

38 -- CI/CD with GitHub Actions

What it is

CI/CD stands for two complementary practices:

- **Continuous Integration (CI)** -- every time you push code or open a pull request, an automated system runs your full test suite, linter, and build. If anything fails, you know immediately -- before the code merges.
- **Continuous Deployment (CD)** -- after the CI checks pass, the system automatically deploys your code to a staging or production environment. No manual "build and upload" step.

GitHub Actions is GitHub's built-in CI/CD platform. You define workflows in YAML files under `.github/workflows/`. Each workflow triggers on specific events (push to a branch, pull request opened, manual dispatch) and runs one or more **jobs** inside disposable containers. Each job contains a sequence of **steps** -- install dependencies, run tests, build images, etc.

The key concepts:

Term	Meaning
Workflow	A YAML file that defines the entire automation pipeline
Trigger (<code>on:</code>)	The event that starts the workflow (push, PR, schedule)
Job	A unit of work that runs on its own virtual machine
Step	A single command or action within a job
Service	A container (like Postgres) that runs alongside your job
<code>needs:</code>	Declares that one job depends on another finishing first

Real-life analogy

Think of a factory assembly line with a quality inspector at the end.

Every time a new part comes off the line (a code push), the inspector runs the full checklist: visual inspection (linting), stress test (unit tests),

integration test (does it fit with the other parts?), and final packaging check (Docker build). If any single check fails, the part is **rejected** -- it never reaches the customer.

You would never ship a product without the inspector's stamp of approval. CI/CD is that inspector, but automated and tireless. It runs the exact same checks every single time, at 3 AM on a Saturday just as reliably as Monday morning. No human forgetfulness, no "I'll test it later."

Without CI: a developer pushes broken code, nobody notices until someone else pulls it and their environment breaks. With CI: the push is checked within minutes and the team sees a red X before the code can merge.

Why we used it here

This project has **three independent codebases** that all need to work together:

- 1. **bilingual_etl/** -- Python ETL pipeline (extraction, transformation, embeddings, loading)
- 2. **search_api/** -- Python FastAPI search microservice
- 3. **admin-ui/** -- React + Vite admin panel

Manual testing across all three is error-prone and time-consuming. Someone might fix a bug in the ETL code but accidentally break a Search API test -- and never notice because they only ran ETL tests locally.

Our CI pipeline catches four categories of problems automatically:

Check	What it catches
ruff lint	Python style errors, unused imports, undefined names
pytest (ETL)	Broken extraction, transformation, or loading logic -- with a real Postgres+pgvector database
pytest (API)	Broken search endpoints, query logic, or response schemas -- also with a real database
npm build	TypeScript errors, broken imports, or build failures in the Admin UI
docker compose build	Dockerfile errors, missing dependencies, or broken multi-service orchestration

The CI runs on every push to `main` or `develop` and on every pull request targeting `main`. This means no broken code can sneak into the main branches.

Code walkthrough

The workflow lives at `.github/workflows/ci.yml`. Let's walk through it section by section.

1. Trigger configuration

```
name: CI

on:
  push:
    branches: [main, develop]
  pull request:
    branches: [main]
```

This says: "Run this workflow when someone pushes to `main` or `develop`, or opens a PR targeting `main`." The `name: CI` is what appears in GitHub's Actions tab and on PR status checks.

2. Environment variables

```
env:
  PYTHON_VERSION: "3.10"
  NODE_VERSION: "20"
  POSTGRES_USER: bilingual
  POSTGRES_PASSWORD: bilingual
  POSTGRES_DB: bilingual search
  DATABASE_URL: postgresql://bilingual:bilingual@localhost:5432/bilingual
```

Workflow-level `env:` variables are available to every job. This avoids repeating the Postgres connection string in multiple places. Note the `DATABASE_URL` points to `localhost:5432` -- inside the CI runner, the service container is mapped to `localhost`.

3. Job 1: lint-and-type-check

```
lint-and-type-check:
  name: Lint & Type Check
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-python@v5
```

```

with:
  python-version: ${{ env.PYTHON_VERSION }}

- name: Install Python dependencies
  run: |
    pip install -r requirements.txt
    pip install ruff mypy

- name: Ruff lint
  run: ruff check bilingual_etl/ search_api/ --select E,W,F --ignore E501

- name: Ruff format check
  run: ruff format --check bilingual_etl/ search_api/ || true

```

This is the fastest job -- no database needed. It installs ruff and runs it against both Python codebases. The `--select E,W,F` flags enable error, warning, and pyflakes rules. `--ignore E501` skips the line-length rule (our code has some long lines by design). The format check uses `|| true` so it warns but does not fail the build -- formatting is advisory, not blocking.

4. Job 2: test-etl (with live database)

```

test-etl:
  name: ETL Tests
  runs-on: ubuntu-latest
  services:
    postgres:
      image: pgvector/pgvector:pg16
      env:
        POSTGRES_USER: ${{ env.POSTGRES_USER }}
        POSTGRES_PASSWORD: ${{ env.POSTGRES_PASSWORD }}
        POSTGRES_DB: ${{ env.POSTGRES_DB }}
      ports:
        - 5432:5432
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5

```

This is where GitHub Actions shines for our project. The `services:` block spins up a **real pgvector container** alongside the test runner. This is not a mock -- it is actual PostgreSQL 16 with the pgvector extension, the same image we use in production (`pgvector/pgvector:pg16`).

The `options:` block configures a health check. GitHub Actions will wait until `pg_isready` succeeds before starting the job steps. This

prevents "connection refused" errors from tests starting before Postgres is ready.

The test step itself:

```
- name: Run ETL tests
  env:
    GEMINI_API_KEY: "test-key-not-used"
  run: |
    pytest tests/etl/ -v --tb=short \
      -k "not test integration" \
      --ignore=tests/etl/test_embeddings.py \
      --ignore=tests/etl/test_llm_provider.py \
      --ignore=tests/etl/test_embedding_provider.py
```

Notice the `--ignore` flags: we skip tests that require real API keys (embedding providers, LLM providers). CI uses a dummy `GEMINI_API_KEY`. The `-k "not test integration"` further filters out slow integration tests. This keeps CI fast while still catching logic errors.

Before tests run, the schema is initialized: `python -m bilingual_etl.load.db` creates the tables and enables pgvector.

5. Job 3: test-api

```
test-api:
  name: Search API Tests
  runs-on: ubuntu-latest
  services:
    postgres:
      # ... same pgvector service as test-etl
  steps:
    # ... checkout, setup-python, install deps, init DB schema
    - name: Run API tests
      run: pytest tests/search_api/ -v --tb=short
```

Same pattern as the ETL tests -- a real pgvector service, schema initialization, then pytest. This job runs the Search API tests (endpoint tests, query logic, response validation). It runs **in parallel** with `test-etl` because neither declares `needs:` on the other.

6. Job 4: test-admin-ui

```
test-admin-ui:
  name: Admin UI Build
  runs-on: ubuntu-latest
  steps:
```

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
with:
node-version: \${ env.NODE_VERSION }
- name: Install dependencies
working-directory: admin-ui
run: npm ci
- name: Build
working-directory: admin-ui
run: npm run build

No database needed here -- this is a pure frontend check. `npm ci` does a clean install (deterministic, uses `package-lock.json` exactly). `npm run build` runs the Vite production build. If there are TypeScript errors, broken imports, or missing dependencies, this step fails. The `working-directory: admin-ui` tells GitHub Actions to run these commands inside the `admin-ui/` subdirectory.

7. Job 5: docker-build (the gate)

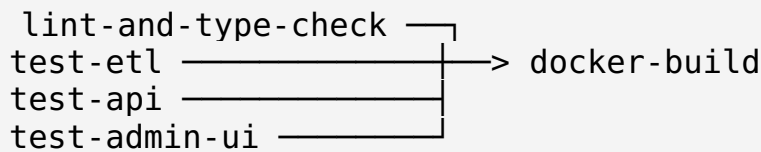
```
docker-build:  
  name: Docker Build  
  runs-on: ubuntu-latest  
  needs: [lint-and-type-check, test-etl, test-api, test-admin-ui]  
  steps:  
    - uses: actions/checkout@v4  
  
    - name: Build Docker images  
      run: docker compose build --no-cache
```

This is the final gate. The `needs:` key is the critical piece -- it lists all four previous jobs. GitHub Actions will **only run this job if all four dependencies pass**. If any single test job or lint job fails, this job is skipped.

`docker compose build --no-cache` builds all three service images (ETL, API, Admin UI) from scratch. This catches Dockerfile issues that might not surface in the individual test jobs (missing system packages, incorrect COPY paths, etc.).

Dependency graph

Here is how the jobs relate:



Jobs 1-4 run **in parallel** (no dependencies between them). Job 5 waits for all four. This means the total CI time is roughly $\max(\text{job1}, \text{job2}, \text{job3}, \text{job4}) + \text{job5}$, not the sum of all five.

How to explain this in an interview / to a teammate

"We use GitHub Actions for CI on this project. Every push to main or develop triggers a workflow that runs five parallel-then-sequential jobs: linting with ruff, ETL tests against a real pgvector database, Search API tests against the same kind of database, an Admin UI production build, and finally a full Docker Compose build that only runs if everything else passes. The key design choice is using GitHub Actions service containers to spin up real Postgres with pgvector rather than mocking the database -- this catches real SQL and vector query issues in CI, not just in production. The dependency graph ensures we never build Docker images from code that has not passed all checks."